



BUILD ON ENTIRE GRAPH — PROJECT DOCUMENT

recipe-finder

Turn an end goal into a proven, *cited* integration recipe — grounded in how real repositories actually wire the code.

What it is A goal-driven engine that returns a playbook where every step has a real `file:line` receipt — evidence mined from the Entire Graph, not an LLM guess.

Live demo recipe-finder-2wvbem0sj-pratik2ts-projects.vercel.app · **Code** [integrations/recipe-finder/](#) · **Skill** [skills/recipe-finder/](#)

1 · Overview

recipe-finder answers the question a developer asks constantly — *"I want to build X; which packages do I use, and how do I wire them together?"* — using the structure of real codebases rather than blog posts, stale docs, or model guesses.

You state the **end goal**, not a library. The Entire Graph — a precomputed, deterministic, local, no-egress code graph of functions, types, and the `IMPORTS` / `CALLS` / `CONSTRUCTS` relations between them — resolves which library real repositories actually use and how they wire it. The output is a ranked playbook in which **every claim is a graph edge with a clickable receipt**. "8 of 10 collaborative-editor repos import `@tiptap/react`" is a fact you can click into, not an opinion.

2 · The problem, and why the graph

Integration advice is ungrounded. Tutorials rot, libraries drift, and a "plausible" API is often the wrong one. The one thing that *is* grounded is production code: what real projects import and call. A code graph turns that into queryable evidence, so a recommendation can be **verified** rather than trusted on faith. That verifiability is the entire value proposition.

3 · How it works

A single engine drives the pipeline. Discovery is grounded in reality; resolution and mining are grounded in the graph.

```
goal -> discover real repos      (GitHub search – grounded, not a model)
      -> entire graph edges      (IMPORTS / CALLS / CONSTRUCTS as NDJSON)
      -> resolve the LIBRARY      (the dominant import repos actually use)
      -> mine the real APIs       (top symbols co-located with that library)
      -> cited recipe            (Install + ranked steps, each with receipts)
```

The graph even corrected our own design. For "kanban", naive auto-resolution first surfaced an icon library; the `IMPORTS` edges showed that real repos use `react-dnd`. Mining `@dnd-kit/sortable` then produced `useSortable` and `useSensor` from real call sites, with receipts — the exact hooks a human would otherwise guess at.

4 · The graph is evidence, not an oracle

This is the hard part, and the core design principle. Static analysis cannot fully resolve dynamic dispatch, reflection, generated code, or inventory-only languages — and some files fail to parse. The product must never present incomplete evidence as certain. So **every step is graded into one of three honest tiers**:

Tier	Meaning	What to do
✓ Confirmed	The graph resolved this edge to a real definition / import.	Safe default — still pin to your installed version.
≈ Heuristic	Matched by name or inference; dynamic dispatch can fool it.	Likely right — open a receipt and confirm against source.
? Unverified	The graph could not resolve it — a blind spot, not an absence.	Treat as a lead; confirm with source or a test.

Each recipe also carries an **analysis** block. It reads the snapshot's completeness signals (parse failures, inventory-only languages, degraded completeness), flags the result as **partial** when the mined set was incompletely analyzed, and prints a concrete **verification path**. A missing step is never treated as "not needed." Crucially, fully-resolved code still yields a complete, confident recipe — the honesty layer adds a caveat, it does not blank out real evidence.

Why it matters: this is what separates a trustworthy tool from a confident-but-wrong one. Users and agents can tell apart confirmed structural facts, heuristic name-matches, and claims that still require a source or test check.

5 · Architecture

THREE SURFACES OVER ONE ENGINE

- › **Skill / CLI** — a coding agent pulls a recipe mid-task. The agent *is* the resolver: it supplies judgment (which library, which repos), the engine supplies evidence.
- › **Web dashboard + API** — a Next.js app and `GET /api/recipe` for humans and programmatic callers, served from a mined corpus.
- › **Docs** — the verify-first angle, in the repo and on the site.

DEPTH

- › **Multi-language.** The engine is language-agnostic (per-language package resolution, install command, and API filtering) across every language Entire Graph resolves semantically — JS/TS, Python, Java, Go, Ruby, Rust, PHP, C#, Kotlin, Swift, and more — keyed off resolved external symbols so internal-method noise is excluded.
- › **Optional LLM resolver.** For the agent-less surfaces (web app, human CLI), Claude can *propose* candidate libraries and search terms; the graph still *verifies* — a candidate is used only if real repos actually import it, so a wrong guess cannot invent a library. It is off unless `ANTHROPIC_API_KEY` is set, and each recipe records the LLM's `suggested` vs the graph's `confirmed` library for transparency.
- › **Databricks Spark scale path.** Raw graph edges land in a `code_edges` Delta table and recipes are aggregated in SQL, so a goal can be mined from many more repos than one Node process could hold.

6 · How it uses Entire Graph

- › **Relationship analysis** — `entire graph edges --relation IMPORTS, CALLS, CONSTRUCTS` is the evidence behind every step.

- › **Search / definition lookup** — used to ground the integration in the real NDJSON producer rather than a guess.
- › **Snapshot completeness** — the summary record (`partial_failures` , `language_tiers` , `completeness_level`) powers the honesty layer.
- › **Semantic diff** — run at submission time to show the entity-level change set.

Throughout, graph results are treated as **evidence, verified against source — never an oracle.**

7 · Running it

```
cd integrations/recipe-finder
npm install                # first time (web app deps)

# Mine a recipe from a goal (needs: entire CLI + graph plugin, gh authed, git, node)
node engine/cli.mjs generate "drag and drop kanban board" --json
node engine/cli.mjs generate "numpy array math" --package numpy --json

npm run dev                # web app at http://localhost:3000
node --test engine/evidence.test.mjs # evidence-grading test suite
```

8 · Status & limitations

- › **Live on Vercel** for browsing the mined corpus. **Live mining does not run on serverless** — it needs the `entire` parser plus `gh` and `git clone` — so a new-goal request returns an honest `501 "live mining unavailable on this host"`. Live mining runs via the skill / local engine, a non-serverless host, or the Databricks path.
- › **Auto-resolution is grounded but can be blunt** on vague goals; mitigated by the agent's `--package` / `--repos` and the optional LLM resolver, not fully solved.
- › **Tests green** — evidence grading and partial-analysis detection are covered by a hermetic suite plus a live end-to-end check when `entire` is present.

In one line: recipe-finder uses the structure of real codebases to support a useful, verifiable decision — which packages to use and how to wire them — and it is honest about exactly how much of that evidence the graph could actually confirm.